```
class DeepLearningEngine:
```

# The Math & Tools

Tensors, Matrices & PyTorch

# What's Been Hiding?

```
// Class 2: Neural Networks
weights * inputs + bias
backprop(gradients)
```

```
// Class 3: Transformers
Q = X * Wq
attention = softmax(Q * K.T) * V
```

What were ALL of these, really?

Matrix Multiplication. That's the secret.

# Today's Journey:

**01** 🧰 Scalars → Vectors → Matrices → Tensors

*// What are these things? Why do they matter?*

**02** 🔮 Matrix Magic

*// What can matrices DO? (Transformations)*

**03** 🔥 PyTorch

*// The tool that makes it all fast, automatic, and beautiful*

# const Scalar = "The Simplest Thing";

```
// Definition

Scalar is just a single number.


let value = 42;
```

```
// Real-world Examples

temperature = 72; // °F

user_age = 25;

coffee_price = 4.99;


// One value. One measurement.
```

Dimensions: 0 *(A single point in space)*

# A Vector — Numbers with Direction

A vector is a list of numbers.

```
location = [28.6139, 77.2090] # lat, long

rgb_color = [255, 87, 51] # red, green, blue

movement = [3, 4] # 3 right, 4 up
```

Dimensions: 1

# Remember these?

```
// 1. Word Embedding

king_vector = [
    0.2, -0.5, 0.8, 0.1, ...
] // 768 dimensions!

"King" as a point in space
```

```
// 2. Input to Neuron

features = [
    x₁, x₂, x₃
]

Features fed into a
neural network
```

These were **vectors** all along.

# class Matrix:

A matrix is a grid of numbers arranged in rows and columns.

> *// Properties*
> - Two-dimensional structure
> - Indexed by (row, col)

📐 Dimensions: 2

*// 2D Tensor Representation*

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

# Matrices You've Already Seen

*// Class 2: Neural Networks*

```
W = WeightMatrix()
output = W × input

# Transforms one vector into another
```

Weights = Matrices

*// Class 3: Transformers*

```
Q = input × Wq
K = input × Wk
V = input × Wv

scores = Q × K.T
```

Attention = Matrix Math

# type Tensor = "The Generalization";

**Scalar** → Just a number → 0D Tensor

**Vector** → A list of numbers → 1D Tensor

**Matrix** → A grid of numbers → 2D Tensor

**Tensor** → Cube / Hypercube → 3D, 4D, 5D...

---

*// It's not a scary new concept.*
*// It's just: "What if we keep adding dimensions?"*

# Why Tensors in Deep Learning?

```
// 1. A single sentence: "The cat sat"
sentence = Matrix(3 tokens, 768 dims)
Shape: [3, 768] # 2D Tensor
```

```
// 2. A batch of sentences
batch = Stack(8 sentences)
Shape: [8, 3, 768] # 3D Tensor
```

```
// 3. Multi-head attention processing
attention = Split(12 heads)
Shape: [8, 12, 3, 64] # 4D Tensor
```

We need tensors because our data has structure.

# const Shape = Everything;

Shape tells you how many dimensions exist and how big they are.

*// Inspecting multi-head attention tensor*

**[8, 12, 3, 64]**

| ↑ | ↑ | ↑ | ↑ |
|---|---|---|---|
| Batch Size | Attention Heads | Number of Tokens | Head Dimension |

*// Pro-tip:* If you understand shape, you can read any model.

# function MatrixTransformation()

A matrix isn't just storage.

**A matrix is a TRANSFORMATION.**

It takes a vector in →

and gives a different vector out.

*// It's a machine that processes data.*

*// Capabilities:*

- Rotate
- Scale
- Stretch
- Reflect
- Project
- Transform meaning

# Matrix × Vector = Transformation

*// Input vector: [1, 0] (pointing right)*

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

*// Output vector: [0, 1] (pointing up!)*

The matrix rotated the vector 90°.

# class ScalingTransform:

*// Scenario 1: Uniform Scaling (2x)*

$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

Every component doubled!
The matrix scaled the vector by 2x.

*// Scenario 2: Stretch & Squish*

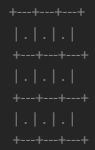
$$\begin{bmatrix} 3 & 0 \\ 0 & 0.5 \end{bmatrix} \times \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Stretched horizontally (3x),
squished vertically (0.5x)!

# view GridTransformations

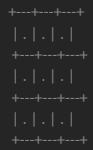
*// Same grid. Different matrices. Different transformations.*

1. Original Grid



2. Rotated Grid

3. Stretched Grid

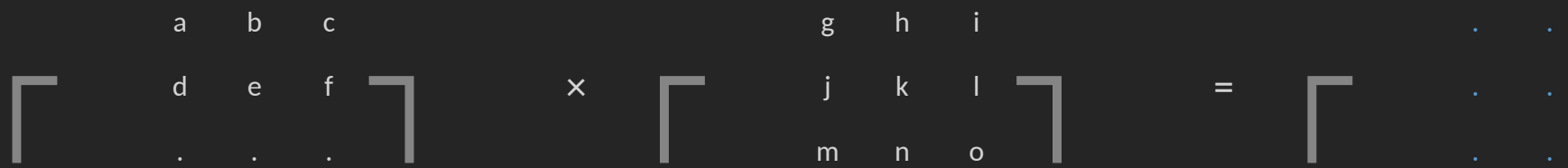


4. Sheared Grid

Key Insight: In a neural network, the model learns which matrix to use.

*// Training is just finding the right transformation for your data.*

# Matrix Multiplication — The Core Operation



# Why Matrix Multiplication is EVERYTHING

| | |
|---|---|
| Forward Pass: | X @ W *// Learned transformations* |
| Backpropagation: | dL/dY @ W.T *// Computing gradients* |
| Attention: | Q @ K.T *// Measuring similarity* |
| Word Embeddings: | One-hot @ W_embed *// Looking up meaning* |

Every single computation in a modern AI model is essentially a series of **matrix multiplications**.

# Why GPUs Exist (For This Exact Reason)

## CPU Architecture

cores: 8-16 "Complex"

optimized_for: SequentialTasks

*// Great for logic, branching, and single-threaded speed.*

## GPU Architecture

cores: 5,000+ "Simple"

optimized_for: ParallelTasks

*// Great for doing the same simple math thousands of times at once.*

Matrix multiplication is "embarrassingly parallel."

*// Every cell in the output can be calculated at the same time.*

# The Dot Product — The Atomic Operation

*// Vectors in space*

v1 = [1, 2, 3]

v2 = [4, 5, 6]

*// The calculation*

v1 · v2 = (1*4) + (2*5) + (3*6) = **32**

*// Conceptual Meaning:*

## Dot Product = Similarity

High value → pointing in the same direction

Low/Zero → pointing in different directions

# Connecting It Back — Attention is Just Dot Products

```
// Input sentence: "The cat sat"

Q_sat · K_the = 0.1

Q_sat · K_cat = 0.8 // ← high similarity!

Q_sat · K_sat = 0.9


// What's happening under the hood?

score = softmax(Q @ K.T)

// The W matrices rotate vectors into a space where
// "relevant" and "similar direction" mean the same thing.
```

Attention scores are **dot products**. The model **learns** the right rotation.

# log.info("Mathematical Foundations Complete")

- [x] Tensors — *// Containers for multi-dimensional data*
- [x] Matrices — *// Transformations (The Machines)*
- [x] MatMul — *// The Engine of Deep Learning*
- [x] Dot Product — *// Measuring Similarity*
- [x] GPU — *// The Parallel Accelerator*

Next: Act 3 — The Tool (PyTorch)

# How PyTorch Actually Works Inside

When you create a tensor, PyTorch stores 2 thing:

- the actual number
- the metadata(shape, device etc)

This metadata tells PyTorch, how to interpret the data

How does `.backward()` actually work? When you set `requires_grad=True` on a tensor, you're telling PyTorch: 'record everything that happens to this tensor.' PyTorch starts rolling a tape.

# Why `torch.no_grad()` and `zero_grad()` Exist

torch.no_grad() : tells PyTorch stop recording

optimizer.zero_grad() : PyTorch *accumulates* gradients by default. If you call `.backward()` twice without clearing, the gradients ADD UP.

# import torch

## What is PyTorch?

PyTorch is the standard tool of modern AI research.

*// Used by:*

- OpenAI
- Anthropic
- Google DeepMind
- Tesla

*// Core Capabilities:*

**01.** Tensors on GPUs

*// 100x speedup over standard CPU math*

**02.** Autograd

*// Automatic differentiation (No more manual backprop)*

**03.** nn.Module

*// High-level building blocks for any model*

If deep learning is the car, PyTorch is the engine and dashboard.

## main.py — Tensors in PyTorch

```python
import torch

# A scalar (0D Tensor)
x = torch.tensor(5.0)

# A vector (1D Tensor)
v = torch.tensor([1.0, 2.0, 3.0])

# A matrix (2D Tensor)
m = torch.tensor([[1, 2], [3, 4]])

# A 3D tensor (e.g., Batch x Height x Width)
t = torch.zeros(2, 3, 4) # shape [2, 3, 4]

# Check shape — you'll do this ALL the time
print(t.shape) # torch.Size([2, 3, 4])
```

# The Magic — GPU Acceleration

```
// 1. Standard CPU Tensor

x = torch.randn(1000, 1000)


// Operations happen on your CPU

// (8-16 cores)
```

```
// 2. Moving to the GPU

device = "cuda" if torch.cuda.is_available() else "cpu"

x = x.to(device) // ← THE MAGIC LINE


// Operations happen on your GPU

// (5,000+ cores)
```

**Result: Up to 100x speedup for large matrix math.**

# Matrix Multiply in PyTorch — The @ Operator

```
// 1. Initialize two matrices
A = torch.randn(3, 4) # Shape: [3, 4]
B = torch.randn(4, 5) # Shape: [4, 5]

// 2. The magic symbol for MatMul
C = A @ B # Shape: [3, 5]

// Or the explicit function:
C = torch.matmul(A, B)
```

The Rule: Inner dimensions MUST match!

*// (3, 4) @ (4, 5) → (3, 5)*

# Tensor Operations — Reshaping

```
// 1. .view() - Reinterpreting data
x = torch.randn(8, 3, 768)
y = x.view(24, 768)

// "Flattening" the batch and tokens
// (8 * 3 = 24)
```

Shape: [8, 3, 768] → [24, 768]

```
// 2. .permute() - Swapping axes
x = torch.randn(8, 12, 3, 64)
y = x.permute(0, 2, 1, 3)

// Swapping Heads and Tokens
// for Attention calculation
```

Shape: [8, 12, 3, 64] → [8, 3, 12, 64]

The Rule: Total number of elements must remain **exactly the same**.

# Broadcasting — PyTorch's Secret Weapon

```
// 1. A matrix and a vector
A = torch.ones(3, 4) # Shape: [3, 4]
B = torch.tensor([1, 2, 3, 4]) # Shape: [4]


// 2. Adding them works! (Wait, how?)
C = A + B # Result Shape: [3, 4]


// PyTorch "stretches" B to match A's shape.
// It's like adding [1,2,3,4] to EVERY row of A.
```

The Rule: Dimensions must be **equal** or one of them must be **1**.

# The Big One — Autograd

## Automatic Differentiation

This is why PyTorch exists. It computes gradients automatically.

*// No more manual calculus.*
*// No more chain rule errors.*

*// How it works:*

**01.** Tracks every operation

*// PyTorch remembers how you got to the result*

**02.** Builds a graph

*// A dynamic computation graph of all tensors*

**03.** Computes gradients

*// One call to .backward() finds all derivatives*

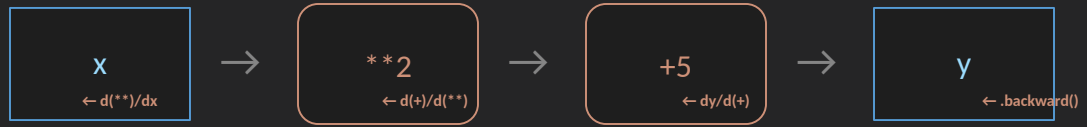
You define the forward pass. PyTorch handles the backward pass.

**autograd_demo.py — Autograd in Action**

```python
import torch

# 1. Create a tensor and tell PyTorch to track it
x = torch.tensor(2.0, requires_grad=True)

# 2. Define a simple function: y = x² + 5
y = x**2 + 5

# 3. Compute gradients automatically!
y.backward()

# 4. Inspect the derivative: dy/dx = 2x = 2(2) = 4
print(x.grad) # Output: tensor(4.0)



// The Magic: One call to .backward() computes everything.
```

# visualize ComputationGraph

*// PyTorch builds this graph dynamically as you run your code.*




Backpropagation is just a reverse traversal of this graph.

*// It applies the chain rule at every step to find how each input affected the output.*

# From Manual to Automatic — Side by Side

```
// 1. Manual Backprop (Painful)

d_loss_y = 2 * (y_pred - y_true)
d_y_relu = 1 if z > 0 else 0
d_relu_w = x.T
d_loss_w = d_relu_w @ (d_loss_y * d_y_relu)
w -= lr * d_loss_w

// One mistake here = Model never trains.
```

```
// 2. PyTorch Autograd (Magic)

loss = criterion(y_pred, y_true)
loss.backward() // ← ALL OF THE LEFT IN ONE LINE
optimizer.step()
```

Reliable. Fast.
Scalable.

**Focus on the Architecture, not the Calculus.**

## model.py — Building with nn.Module

```python
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x
```

`nn.Module` is PyTorch's way of packaging a model. You declare layers in `__init__` — each one is a matrix (a transformation). You define how data flows in `forward`. That's the entire model.

`nn.Linear(784, 128)` creates a 128×784 matrix. When data passes through, it does `input @ weight.T + bias`.

Every model follows this exact pattern.

*// Define layers in __init__, connect them in forward.*

# What's Inside nn.Linear?

```
// Define a linear layer: 3 inputs → 2 outputs
layer = nn.Linear(3, 2)

// Let's peek inside:


layer.weight # Shape: [2, 3]
[

[0.12, -0.45, 0.89],
[-0.34, 0.22, -0.11]
]
layer.bias # Shape: [2]
[0.05, -0.02]
```

It's just **Matrix Multiplication** and **Vector Addition**.

*// y = x @ W.T + b*

## train.py — The 5-Line Pattern

```
for epoch in range(10):
    # 1. Forward Pass
     y_pred = model(x_batch)

    # 2. Compute Loss
     loss = criterion(y_pred, y_batch)

    # 3. Clear Old Gradients
     optimizer.zero_grad()

    # 4. Backward Pass (The Calculus)
     loss.backward()

    # 5. Update Weights
     optimizer.step()

// This pattern is the "heartbeat" of every DL model.
 From MNIST to GPT-4, this loop remains the same.
```

# The 5 Lines, Decoded

| | | |
|---|---|---|
| 1. | Forward | *// Predict using current weights. "How did we do?"* |
| 2. | Loss | *// Measure the error. "How wrong were we?"* |
| 3. | zero_grad | *// Flush the toilet. Don't let old gradients leak.* |
| 4. | Backward | *// The Calculus. Find how each weight caused the error.* |
| 5. | Step | *// Update weights. Nudge them in the right direction.* |

Remember: If you miss step 3, gradients accumulate and your model explodes.

# Putting It All Together — Live Demo

bash — 1280×720

```
ubuntu@manus:~$ python3 train_mnist.py

[INFO] Loading MNIST dataset...
[INFO] Initializing SimpleNet (784 -> 128 -> 10)...
[INFO] Starting training loop on CUDA...

// We are about to watch:
1. Data becoming Tensors
2. Matrices performing Transformations
3. Autograd finding the errors
4. The model Learning to see
```

**Goal: 98% Accuracy in under 60 seconds.**

# What Just Happened? — The Transformation Recap

| | | |
|---|---|---|
| 1. | Flattening | *// [28, 28] image grid → [784] list of pixels.* |
| 2. | Transform 1 | *// [784] @ [784, 128] → [128] features.* |
| 3. | Activation | *// ReLU: Filtering out the noise.* |
| 4. | Transform 2 | *// [128] @ [128, 10] → [10] digit scores.* |

Training is just finding the **perfect matrices** to map
"image pixels" → "correct number label".